



DEMO MODEL

OmniMetric - Distance Estimation Model

PANORAMA 360° (Phase 1)

```
Hybrid_Save.py
# --- SETUP ---
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")

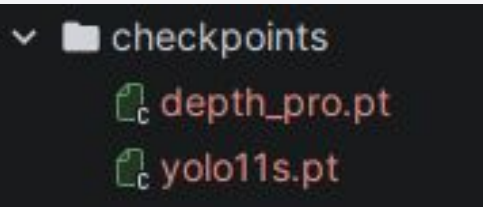
# Load Models
print("Loading Depth Pro...")
depth_model, depth_transform = depth_pro.create_model_and_transforms()
depth_model.eval().to(device)

print("Loading YOLO...")
yolo_model = YOLO("yolo11s.pt")

image_path = "data/img_001384.jpg"
```

```
Hybrid_Save.py
# =====
# 1. VISUALIZATION LAYER (Direct Run)
# =====
print("\n--- PHASE 1: Generating Visual Background ---")
original_img = cv2.imread(image_path)
if original_img is None:
    raise FileNotFoundError(f"Image not found at {image_path}")

h, w = original_img.shape[:2]
pad_w = 256
padded_img = np.hstack((original_img[:, -pad_w:], original_img))
```



- โหลด Model และ Image_path
- ทำ padding ให้รูปภาพเพื่อให้ รอยต่อของภาพ Panorama ผลลัพธ์จะได้เนียนขึ้น

PANORAMA 360° (Phase 1)

```
Hybrid_Save.py

# Run Depth Pro
img_pil = Image.fromarray(cv2.cvtColor(padded_img, cv2.COLOR_BGR2RGB))
d_input = depth_transform(img_pil).to(device)
f_px_visual = torch.tensor(w / 2.0).to(device) # Estimate Focal Length in Pixels

with torch.no_grad():
    prediction = depth_model.infer(d_input, f_px=f_px_visual)
    depth_vis = prediction["depth"].squeeze().cpu().numpy()
```

```
Hybrid_Save.py

# Crop back & Colorize
depth_vis = depth_vis[:, pad_w:]
depth_vis = cv2.resize(depth_vis, dsize=(w, h))
depth_norm = (depth_vis - depth_vis.min()) / (depth_vis.max() - depth_vis.min())
inv_depth = 1.0 - depth_norm
visual_map = cv2.applyColorMap((inv_depth * 255).astype(np.uint8), cv2.COLORMAP_TURBO)

# Save Phase 1 Result
phase1_filename = "phase1_visual_background.jpg"
cv2.imwrite(phase1_filename, visual_map)
print(f"[SAVED] Visual Background saved to: {phase1_filename}")
```

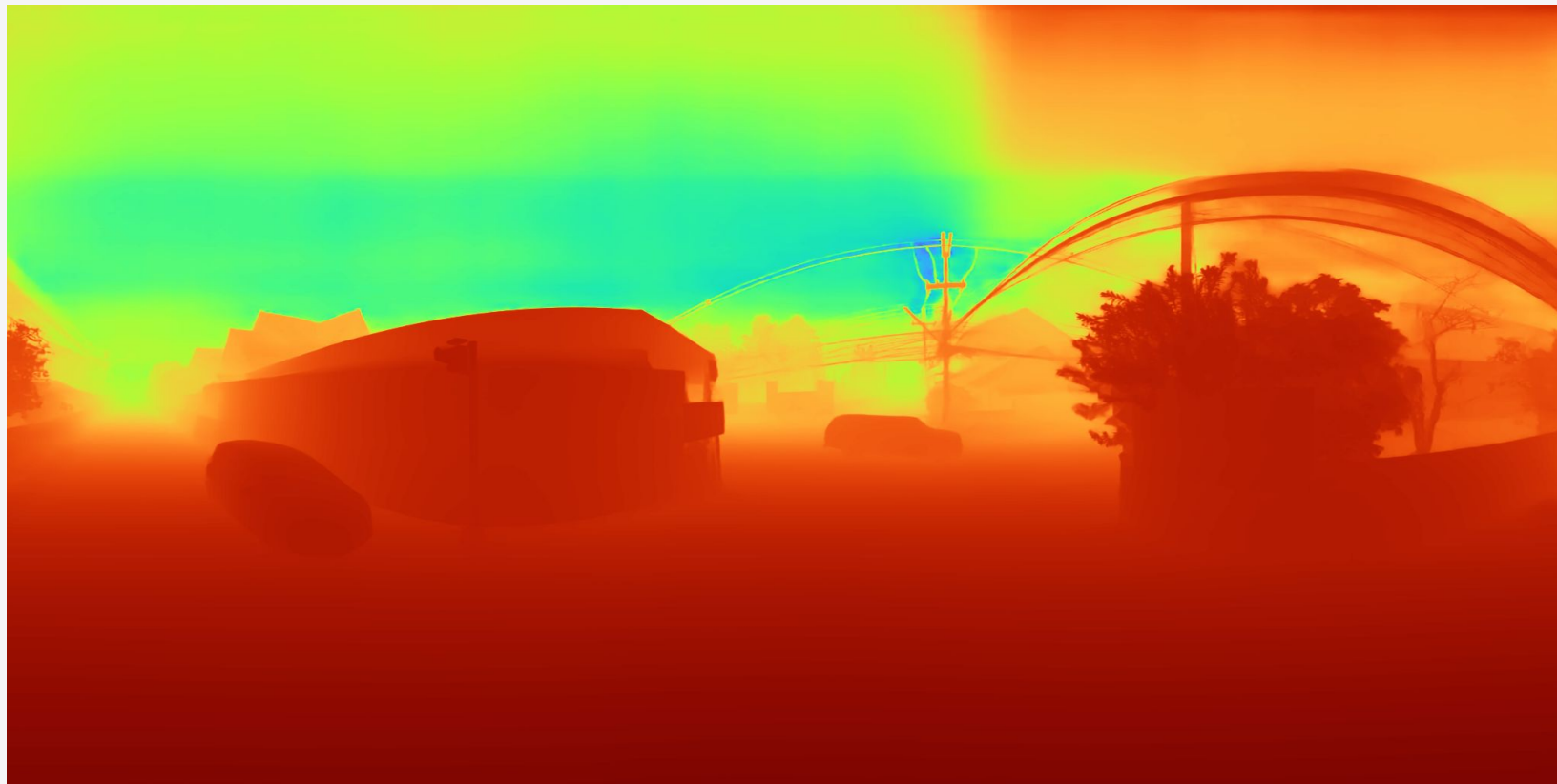
1. การประมวลผล (Run Depth Pro)

- **เตรียมภาพ:** แปลงไฟล์ภาพจาก OpenCV (BGR) เป็น RGB เพื่อให้โมเดล AI ของ Apple ประมวลผลได้ถูกต้อง
- **คำนวณระยะ:** กำหนดค่า `f_px_visual` (Focal Length) เพื่อบอกมุมมองกล้อง ช่วยให้ AI ประเมินระยะห่างในภาพ Panorama ได้แม่นยำ
- **ถอดรหัสความลึก:** ใช้คำสั่ง `depth_model.infer` เพื่อเปลี่ยนพิกเซลในภาพให้กลายเป็นตัวเลขระยะทางจริง (หน่วยเป็นเมตร)

2. การสร้างภาพจำลอง (Colorization & Visualization)

- **ปรับขนาด:** ตัดส่วนที่เกินออกและ Resize ภาพให้กลับมาเท่าขนาดต้นฉบับเพื่อให้ตำแหน่งแม่นยำ
- **ใส่สีมิติ:** แปลงตัวเลขระยะทางให้กลายเป็นสีแบบไล่เฉดสีเพื่อให้คนดูเข้าใจง่าย
 - สีโทนร้อน (แดง/ส้ม): คือวัตถุที่อยู่ ใกล้
 - สีโทนเย็น (น้ำเงิน/เขียว): คือวัตถุที่อยู่ ไกล
- **บันทึกผล:** เซฟเป็นไฟล์ภาพ phase1 เพื่อใช้เป็นตัวแสดงผล

PANORAMA 360° (Phase 1)



PANORAMA 360° (Phase 2)

```
Hybrid_Save.py
# =====
# 2. DETECTION & METRIC LAYER (Sniper Method)
# =====
print("\n--- PHASE 2: Sniper Logic Execution ---")

results = yolo_model(original_img, device=0 if device == "cuda" else "cpu", verbose=False)
final_output = visual_map.copy()

object_counter = 0
```

```
Hybrid_Save.py
for result in results:
    boxes = result.bboxes.xyxy.cpu().numpy()
    classes = result.bboxes.cls.cpu().numpy()
    confs = result.bboxes.conf.cpu().numpy()
```

```
Hybrid_Save.py
for box, cls, conf in zip(boxes, classes, confs):
    label = result.names[int(cls)]

    if label in ['car', 'truck', 'person', 'bus']:
        object_counter += 1
        x1, y1, x2, y2 = map(int, box)
        cx, cy = (x1 + x2) // 2, (y1 + y2) // 2

        print(f"\n[Object #{object_counter}] DETECTED: {label.upper()}")
        print(f" > YOLO Confidence : {conf:.4f} ({(conf * 100):.1f}%)")
```

1. รันโมเดล YOLO กับภาพต้นฉบับ
2. วนรูปไปใน results เพื่อรับค่า Box,cls,conf
3. วนรูปดู "วัตถุแต่ละชิ้น" ในรูปใบนั้น และหาค่าจุดกึ่งกลางของ Bounding Box

PANORAMA 360° (Phase 2)

```
Hybrid_Save.py

# 1. Convert to Angle
yaw_deg = (cx / w) * 360 - 180
pitch_deg = -((cy / h) * 180 - 90)

# 2. Generate Crop
target_fov = 80
sniper_face_size = 512

perspective_crop = py360convert.e2p(
    original_img,
    fov_deg=target_fov,
    u_deg=yaw_deg,
    v_deg=pitch_deg,
    out_hw=(sniper_face_size, sniper_face_size),
    in_rot_deg=0,
    mode='bilinear'
)

# [SAVED] Debug RGB Crop
crop_filename = f"debug_rgb_crop_{object_counter}_{label}.jpg"
cv2.imwrite(crop_filename, perspective_crop)
print(f" > [SAVED] RGB Crop saved: {crop_filename}")
```

```
Hybrid_Save.py

# 3. Metric Measurement
f_px_crop_val = (sniper_face_size / 2.0) / np.tan(np.deg2rad(target_fov) / 2.0)

crop_pil = Image.fromarray(cv2.cvtColor(perspective_crop, cv2.COLOR_BGR2RGB))
crop_tensor = depth_transform(crop_pil).to(device)
f_px_tensor = torch.tensor(f_px_crop_val).to(device)

with torch.no_grad():
    pred_crop = depth_model.infer(crop_tensor, f_px=f_px_tensor)
    depth_crop = pred_crop["depth"].squeeze().cpu().numpy()
```

1. คำนวณ "มุมมองศา" (ทิศทางซ้าย-ขวา และ ก้ม-เงย) เพื่อให้ระบบรู้ว่าจะต้อง "หันกล้อง" ไปทางไหน
2. ตั้งค่าการซูม (FOV)
3. ตัดภาพมุมมองปกติ (Perspective Crop) และบันทึกผลเพื่อตรวจสอบ
4. คำนวณค่า Focal Length ใหม่ (Recalculate)
5. ทำ Preprocessing ของแต่ละภาพ
6. `depth_model.infer` คำนวณความลึกอีกครั้ง เฉพาะในส่วนที่ซูมมา

PANORAMA 360° (Phase 2)

```
Hybrid_Save.py
center_crop_y = sniper_face_size // 2
center_crop_x = sniper_face_size // 2

# 3.1 Visualization Depth Crop
d_inv = 1.0 / (depth_crop + 1e-5)
d_norm = (d_inv - d_inv.min()) / (d_inv.max() - d_inv.min())
d_vis_uint8 = (d_norm * 255).astype(np.uint8)

depth_crop_colored = cv2.applyColorMap(d_vis_uint8, cv2.COLORMAP_TURBO)

cv2.circle(depth_crop_colored, center: (center_crop_x, center_crop_y), radius: 5, color: (0, 255, 0), -1)

# Save File
depth_filename = f"debug_depth_crop_{object_counter}_{label}.jpg"
cv2.imwrite(depth_filename, depth_crop_colored)
print(f" > [SAVED] Depth Crop saved: {depth_filename}")
```

```
Hybrid_Save.py
# 3.2 Metric Calculation
metric_dist = np.median(depth_crop[center_crop_y - 2:center_crop_y + 2,
center_crop_x - 2:center_crop_x + 2])

# ----- Alt+F Finish changes to file | Escape Dismiss
print(f" > \ Measured Distance : {metric_dist:.4f} meters")
# Draw on Final Output
text = f"{label}: {metric_dist:.2f}m"
cv2.rectangle(final_output, (x1, y1), (x2, y2), (255, 255, 255), 2)
cv2.putText(final_output, text, org: (x1, y1 - 10),
cv2.FONT_HERSHEY_SIMPLEX, fontScale: 0.8, color: (255, 255, 255), thickness: 2, cv2.LINE_AA)
```

1. หาจุดกลางของรูปที่ crop
2. แปลงเป็นภาพสี depth map
3. วาดจุดกลางของรูป (วัตถุ) และบันทึกภาพเพื่อตรวจสอบ
4. คำนวณระยะทางจริง ดึงค่าความลึกจากพิกเซลรอบๆ จุดกึ่งกลาง แล้วหาค่า มัธยฐาน (Median)
5. วาดกรอบ Box และค่าระยะทาง ลงบนภาพ Panorama หลัก เพื่อแสดงผล

PANORAMA 360° (Phase 2)

